AutoChip Report

Introduction

This assignment explored the use of AutoChip for LLM-assisted Verilog generation and iterative RTL refinement. AutoChip was used to complete eight tutorial examples, followed by an accumulator-based 8-bit CPU design and an additional signed-by-unsigned multiplier problem.

For the tutorial examples, the same general AutoChip configuration was used throughout. The model was gpt-4o-mini through the ChatGPT model family, with five candidate responses generated per iteration. Icarus Verilog was used for compilation and simulation. The provided testbenches were modified when necessary so that AutoChip could read the final number of mismatches and samples.

The general verification output format was:

Mismatches: <number of mismatches> in <number of samples>

The testbenches were also modified so that failures did not terminate simulation immediately, allowing AutoChip to obtain partial scores and use them as feedback.

Example 1 – binary_to_bcd

A. Initial Specification

The first example was a combinational binary-to-BCD converter. The input was a 5-bit binary value, and the output was an 8-bit BCD representation containing a 4-bit tens digit and a 4-bit ones digit.

AutoChip Parameters:

- Model family: ChatGPT
- Model: gpt-4o-mini
- Number of candidates: 5
- Maximum iterations: 10
- Test samples: 32

B. Trajectory

The initial AutoChip candidates produced several partially correct implementations. Their mismatch counts differed significantly, which showed that the testbench was able to provide meaningful partial feedback.

The candidate ranks in iteration 0 included values such as 0.3125, 0.1875, and 0.25. One candidate correctly implemented the conversion and produced zero mismatches.

C. Final Verification

The best candidate achieved:

Mismatches: 0 in 32 samples
Best rank: 1.0
Best iteration: 0

D. Discussion

This example demonstrated the usefulness of quantitative testbench feedback. Several candidates were only partially correct, while one candidate successfully implemented the Binary-to-BCD conversion in the first iteration.

Example 2 – sequence_detector

A. Initial Specification

The sequence detector accepts a 3-bit value on each clock cycle and detects the sequence:

001
101
110
000
110
110
011
101

The output sequence_found should assert when the complete sequence is detected.

AutoChip Parameters:

- Model family: ChatGPT
- Model: gpt-4o-mini
- Number of candidates: 5
- Maximum iterations: 10
- Test samples: 12

B. Trajectory

The first testbench adaptation initially caused simulation to run indefinitely because the clock-generation process never terminated. The clock

logic was changed so that the clock process stopped cleanly after all samples were evaluated.

After this correction, AutoChip generated candidates with different levels of correctness. Some candidates had one mismatch, another had four mismatches, and one candidate failed compilation.

C. Final Verification

The best candidate produced:

Mismatches: 0 in 12 samples
Best rank: 1.0
Best iteration: 0

The iteration-0 candidate ranks were approximately:

1.0
0.9167
0.9167
0.6667
-1

D. Discussion

This example demonstrated the importance of testbench structure in AutoChip. A nonterminating simulation prevents useful feedback from reaching the model. Once the clock-generation issue was fixed, AutoChip successfully distinguished between partially correct and fully correct implementations.

Example 3 – shift_register

A. Initial Specification

The shift-register design has the following inputs:

- Clock
- Active-low reset
- 1-bit data input
- Shift enable

The output is an 8-bit register. When shift enable is asserted, the register shifts left and inserts the new input bit into the least significant bit. When shift enable is low, the register should hold its value.

AutoChip Parameters:

- Model family: ChatGPT
- Model: gpt-4o-mini
- Number of candidates: 5
- Maximum iterations: 10

B. Trajectory

The first AutoChip-compatible version of the testbench produced the same result of 6 mismatches out of 7 samples for every generated candidate. Because every candidate received the same score, the testbench was not providing useful feedback about differences between implementations.

The testbench was rewritten to directly verify reset behavior, shifting both 0 and 1 values, multiple consecutive shifts, hold behavior when shift_enable was low, and a second reset after state changes.

After correcting the verification method, AutoChip generated a fully correct implementation.

C. Final Verification

The best result was:

Best rank: 1.0
Best iteration: 0
Best response number: 0

The generated implementation used a standard sequential shift register:

```
always @(posedge clk or negedge reset_n) begin
   if (!reset_n) begin
      data_out <= 8'b00000000;
   end else if (shift_enable) begin
      data_out <= {data_out[6:0], data_in};
   end
end
```

D. Discussion

The main issue in this example was not the generated RTL but the scoring mechanism. The identical mismatch count for every candidate revealed that the first testbench adaptation was not useful for AutoChip feedback. Replacing it with direct functional verification made the testbench better suited for iterative LLM-based design.

Example 4 – abro_state_machine

A. Initial Specification

The ABRO example required a state machine with inputs:

- Clock
- Active-low reset
- A
- B

The outputs were the main output O and a one-hot encoded state output used for verification.

AutoChip Parameters:

- Model family: ChatGPT
- Model: gpt-4o-mini
- Number of candidates: 5
- Maximum iterations: 10
- Test samples: 16

B. Trajectory

This was one of the most difficult tutorial examples. In iteration 0, the best candidate achieved a rank of only:

0.4375

As AutoChip received additional simulation feedback, performance gradually improved.
Later iterations reached:

Iteration 7: 0.625
Iteration 9: 0.8125
Iteration 10: 0.8125

The best implementation had three mismatches out of sixteen samples.

C. Final Verification
The best result was:

Mismatches: 3 in 16 samples
Best rank: 0.8125

The design did not fully converge to a correct implementation within the configured iterations.

D. Discussion

This example is a useful case where iterative AutoChip feedback improved the generated RTL without reaching a fully correct solution. The initial natural-language specification was relatively underspecified compared with examples such as the LFSR, making it more difficult for the model to infer the intended state transitions.

Example 5 – dice_roller
A. Initial Specification

The dice roller generates pseudo-random values corresponding to one of four dice:

- 4-sided
- 6-sided
- 8-sided
- 20-sided

A 2-bit input selects the die, and the output must remain inside the valid range for the selected die.

AutoChip Parameters

- Model family: ChatGPT
- Model: gpt-4o-mini
- Number of candidates: 5
- Maximum iterations: 10
- 1000 rolls per die type

B. Trajectory

AutoChip successfully generated a correct implementation immediately in iteration 0.

The best solution used $urandom with modulo operations, for example:

```
rolled_number <= $urandom % 4 + 1;
```

for a four-sided die.

C. Final Verification
The best candidate achieved:

Best rank: 1.0
Best iteration: 0

All generated values tested by the testbench were within the required ranges.

D. Discussion

The dice-roller example showed that AutoChip could handle randomized simulation-oriented behavior effectively. The provided verification checked valid output ranges rather than the statistical quality of the generated distribution.

Example 6 – lfsr
A. Initial Specification

The LFSR was an 8-bit linear-feedback shift register with:

- Initial state: 10001010
- Taps at locations 1, 4, 6, and 7

The expected state transition was derived from the supplied testbench.

AutoChip Parameters:
- Model family: ChatGPT
- Model: gpt-4o-mini
- Number of candidates: 5
- Maximum iterations: 10
- Test samples: 257

B. Trajectory

This was the most consistently successful example. All five candidates generated during iteration 0 passed the complete testbench.

The response ranks were:

[1.0, 1.0, 1.0, 1.0, 1.0]

C. Final Verification
Each candidate produced:

Mismatches: 0
Samples: 257

The best rank was:

1.0

D. Discussion

The LFSR specification was precise, including both the initial state and feedback taps. This appears to have made the design especially easy for AutoChip compared with more ambiguous state-machine examples.

Example 7 – sequence_generator
A. Initial Specification

The sequence generator repeatedly outputs:

AF
BC
E2
78
FF
E2
0B
8D

while enabled. When disabled, it should hold the current value.

AutoChip Parameters:
- Model family: ChatGPT
- Model: gpt-4o-mini
- Number of candidates: 5
- Maximum iterations: 10
- Test samples: 17

B. Initial Compilation Failure

The first AutoChip run failed compilation for every candidate. Inspection of the generated RTL revealed that the model repeatedly used the identifier: sequence for an internal array.

Because AutoChip invokes Icarus Verilog in SystemVerilog mode, sequence is a reserved SystemVerilog keyword. This produced syntax errors in every generated design.

The prompt was modified to explicitly state that the identifier sequence must not be used and recommended alternatives such as seq_mem.

C. Testbench Timing Problem

After fixing the reserved keyword issue, the generated designs compiled but produced approximately 16 - 17 mismatches out of 17 samples.

The testbench was sampling outputs immediately at the positive clock edge. Designs using normal nonblocking sequential assignments had not yet updated their outputs when the testbench performed the comparison.

A small delay was added after each positive edge so that nonblocking assignments could settle before verification.

D. Final Verification

After both corrections, four out of five candidates passed immediately:

Response ranks: [1.0, -0.5, 1.0, 1.0, 1.0]

The best result was:
Mismatches: 0
Samples: 17
Best rank: 1.0

Best iteration: 0

E. Discussion

This example showed the strongest interaction between model generation, compiler feedback, and testbench design.

Two different issues were discovered:

1. Use of a reserved SystemVerilog keyword in generated RTL.
2. A testbench timing issue caused by nonblocking assignment semantics.

After these issues were identified and corrected, AutoChip generated several completely correct implementations.

Example 8 – traffic_light

A. Initial Specification

The traffic-light FSM has red, green, and yellow states.

The required timing is:

- Red for 32 clock cycles
- Green for 20 clock cycles
- Yellow for 7 clock cycles

The sequence then repeats.

AutoChip Parameters:

- Model family: ChatGPT
- Model: gpt-4o-mini
- Number of candidates: 5
- Maximum iterations: 10
- Verification checkpoints: 4

B. Trajectory

The first five generated candidates produced a range of scores:

[0.5, 0.5, 0.5, 0.25, 1.0]

This indicated that several candidates captured only part of the required timing behavior, while one candidate correctly implemented all state transitions.

C. Final Verification

The best candidate achieved:

Mismatches: 0
Samples: 4
Best rank: 1.0
Best iteration: 0
Best response number: 4

D. Discussion

This example showed that even when several candidates were only partially correct, generating multiple candidates allowed AutoChip to find a correct implementation immediately.

Part II – 8-bit CPU

A. Starting Architecture

The CPU experiment used a compact accumulator-based 8-bit processor designed to remain consistent with the small-design constraints of the assignment.

The CPU used:

- 8-bit accumulator
- 4-bit program counter
- 8-bit single-byte instructions
- 4-bit opcode
- 4-bit operand
- Input port
- Output port
- Output-valid signal
- Halt signal

The ISA included:

0x0 NOP
0x1 LDI
0x2 ADD
0x3 SUB
0x4 AND
0x5 OR
0x6 XOR
0x7 SHL
0x8 SHR
0x9 INC
0xA DEC
0xB IN
0xC OUT
0xD JMP
0xE JZ
0xF HALT

All instructions were one byte long.

B. Verification Strategy

A custom test program was constructed to exercise several major parts of the processor:

```
LDI 3
ADD 5
SHL
SUB 1
XOR F
JZ 7
LDI F      ; skipped
IN
INC
OUT
HALT
```

The testbench verified:

- accumulator arithmetic,
- program-counter advancement,
- conditional branching,
- skipped instructions,
- input behavior,
- output behavior,
- out_valid,
- halt behavior, and
- state retention after halt.

C. AutoChip Result

AutoChip successfully generated a fully passing CPU implementation during iteration 0.
The final result was:

Best rank: 1.0
Best iteration: 0
Best response number: 4

The generated CPU correctly used an accumulator and implemented the complete custom ISA. Branch instructions handled the program counter separately from the default PC increment, and execution stopped when halted became asserted.

D. Discussion

The CPU example was expected to be substantially more difficult than the small tutorial circuits. However, the detailed ISA and explicit behavioral specification allowed AutoChip to generate a correct implementation without iterative refinement.

This suggests that specification clarity may be at least as important as raw design complexity. The CPU contained more functionality than the tutorial examples, but its instruction behavior was described explicitly. In contrast, the ABRO state-machine example was much smaller but more difficult because its behavioral specification was less explicit.

A limitation of this result is that the custom test program does not exhaustively test every possible instruction combination or control-flow pattern. A more complete verification environment could include individual directed tests for every opcode, overflow behavior, multiple jumps, untaken conditional branches, enable behavior, and longer programs.

Extra Problem – signed_unsigned_multiplier

A. Specification

The extra problem required a combinational multiplier with:

- 8-bit signed input a
- 8-bit unsigned input b
- 16-bit signed output p

The key challenge was correctly preserving the signed interpretation of a while ensuring that values of b above 127 were still interpreted as positive unsigned values.

B. Verification

An exhaustive testbench evaluated every possible input combination:

a: -128 to 127
b: 0 to 255

This produced:

$256 \times 256 = 65{,}536$

test cases.

C. AutoChip Trajectory

Unlike several tutorial examples, the multiplier required an additional iteration before a fully correct implementation was generated.
The best implementation was found at:

Best rank: 1.0
Best iteration: 1
Best response number: 1

The final design explicitly represented the signed operand at 16-bit width before multiplication:

```
wire signed [15:0] signed_a;
assign signed_a = a;
assign p = signed_a * b;
```

D. Discussion

The exhaustive testbench was particularly useful for this problem because signed/unsigned multiplication can fail only for certain portions of the input range. Testing all 65,536 combinations gave much stronger verification than a small set of directed examples.

The fact that the best solution appeared in iteration 1 also provides another example of AutoChip improving after receiving tool feedback rather than solving the problem in a single generation attempt.

Overall Discussion

Across the assignment, AutoChip performed best when the natural-language specification was explicit and the testbench provided clear quantitative feedback.

Several patterns were observed.

First, multiple candidates were useful even when no iterative correction was required. In the Binary-to-BCD and traffic-light examples, candidate quality varied substantially during iteration 0, but at least one correct solution was generated.

Second, the quality of the verification environment strongly affected AutoChip's ability to improve. The initial shift-register scorer produced identical results for every candidate and therefore provided little useful information. Similarly, the sequence-generator testbench initially sampled sequential outputs too early.

Third, compiler feedback exposed problems that were not directly related to the intended hardware behavior. In the sequence-generator example, the LLM repeatedly selected sequence, a reserved SystemVerilog keyword, as an internal identifier.

Finally, design complexity alone did not determine difficulty. The 8-bit CPU was substantially larger than the ABRO state machine, yet the CPU succeeded in iteration 0 while ABRO did not fully converge. The CPU specification defined the behavior of every instruction explicitly, whereas the ABRO behavior was less completely specified.

Conclusion

AutoChip successfully generated correct RTL for most of the tutorial examples, the 8-bit CPU, and the extra signed-by-unsigned multiplier problem.

The experiments showed that LLM-assisted RTL generation can work well when paired with strong automated verification. However, the results also demonstrated that compiler configuration, testbench timing, scoring quality, and specification ambiguity can strongly affect the generation trajectory.

The most useful aspect of AutoChip was not simply producing Verilog code, but combining generation with compilation and simulation feedback. This allowed partially correct implementations to be evaluated quantitatively and, in several cases, allowed problems in either the RTL or verification environment to be identified and corrected.